
S³: 在生成式推理期间提高 GPU 利用率以提升吞吐量

Yunho Jin
Harvard University

Chun-Feng Wu
National Yang Ming
Chiao Tung University

David Brooks
Harvard University

Gu-Yeon Wei
Harvard University

摘要

使用大型语言模型（LLM）生成文本会消耗大量内存。除了本来就很大的模型参数，用于保存序列中此前 token 信息的键/值缓存（KV 缓存）还可能增长到比模型本身更大的规模。一些现有 LLM 服务框架不知道输出序列的长度，为了保证能够生成完整序列，会按最大序列长度为 KV 缓存预留内存，这进一步加剧了问题。预留内存限制了可用的批大小，使 GPU 利用率下降，尤其降低了吞吐量。我们认为，在系统设计中提前获知输出序列的信息可以缓解这一问题。为此，我们提出 S³：预测输出序列长度，根据预测结果调度生成请求，从而提高设备资源利用率和吞吐量，并处理预测错误。相对于按输出长度的最坏情况预留内存的系统，我们的方法实现了 6.49× 的吞吐量。

1 引言

文本生成在各类服务中日益普及，推动了大型语言模型（LLM）的使用；这类模型以较高的准确率著称。LLM 具有区别于非 Transformer 模型的特征，包括需要大量计算资源和内存资源。具体而言，我们观察到，基于 Transformer 的 LLM 常常受内存容量和带宽限制，计算资源则明显未被充分使用。在 NVIDIA A100 GPU 上运行 GPT-J 服务时，GPU 计算资源利用率可以低至 0.4%。这反映了 LLM 受内存限制的特性，也说明要提高 GPU 计算资源利用率，就需要更有效地使用内存。

提高 GPU 利用率和吞吐量的常见方法是增大批大小。一个批次中的输入共享相同的模型权重，因此 GPU 只需将权重从高带宽内存（HBM）载入片上 SRAM 一次，便可让批次中的所有输入复用这些权重。处理同一组模型权重时，GPU 因而可以使用更多计算资源。增大批大小是一种简单的优化方法，因此在卷积神经网络和全连接神经网络的服务系统中很常见 [1, 2]。

然而，基于 Transformer 的文本生成 LLM 使用自回归方式，其自注意力层使这一简单优化遇到困难。具体来说，在序列中生成一个新 token 时，模型需要关注该序列中此前的所有 token，因此必须保留此前 token 的信息并将其存入 HBM。我们把 HBM 中存储这些信息的区域称为键/值缓存（KV 缓存）。批大小增大、序列变长都会扩大 KV 缓存，进而限制最大批大小，降低 GPU 利用率，最终降低吞吐量。为支持不断增长的 KV 缓存，Huggingface 的 Transformers 库 [3] 在每次生成 token 时不断分配新的内存，并引入与内存分配有关的时延开销。这提高了易用性，因为用户无需预先知道输出长度，但推理时延较长。另一种方法是 NVIDIA 的 FasterTransformer 库 [4] 按最大序列长度预先分配内存；分配量超过实际需要，因而浪费了内存。这种方法限制批大小，并优先考虑时延而非吞吐量。近期模型的最大序列长度不断增加，例如达到 8K 或 32K [5]，这进一步放大了 KV 缓存的开销，要求基于 Transformer 的文本生成服务更有效地使用内存。

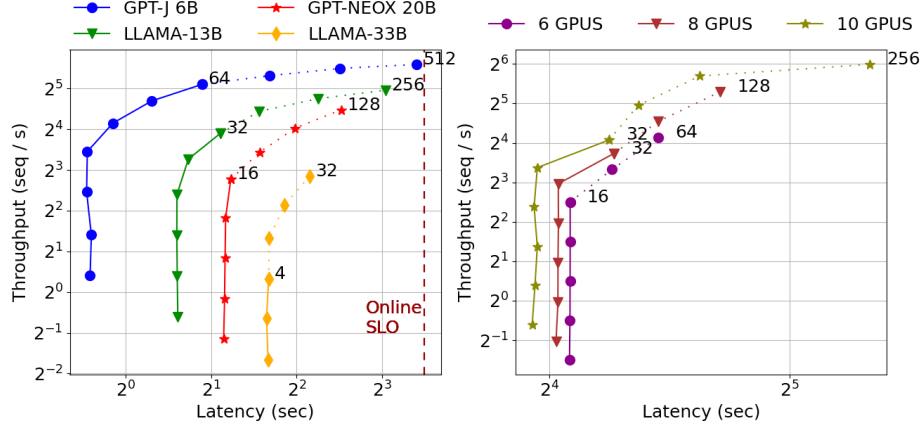


图 1: 生成 60 个 token 时, 不同模型 (左) 和不同 GPU 数量 (右, 将 GPT-3 分布到 6、8、10 张 GPU) 下的时延与吞吐量权衡, 绘图思路参考 FlexGen [6]。曲线上的标记代表批大小, 从 1 开始按 2 的幂递增, 直到一张 A100 能容纳的最大批大小。为每条序列精确分配所需内存, 可以以较高时延为代价, 把曲线扩展到更高吞吐量。实线表示原始系统面对的权衡, 虚线表示 S^3 能够扩展的范围。数字表示 S^3 和原始系统的最大批大小。左图的竖线表示阅读一条 60-token 序列所对应的时延服务等级目标 (SLO)。图内英文: Latency (sec) 为时延 (秒), Throughput (seqs / s) 为吞吐量 (序列/秒), Online SLO 为在线场景的 SLO, GPUS 为 GPU 数量。

针对这些问题, 我们提出 S^3 , 即 scheduling sequences with speculation (基于推测的序列调度)。该框架通过预测输出序列长度、减少内存浪费来最大化吞吐量。Huggingface Transformers 的频繁内存分配和 FasterTransformer 受限的批大小, 都源于缺乏对输出长度的先验知识。 S^3 预测期望的输出长度, 并为每个请求分配相应数量的内存, 同时调度序列以提高 GPU 利用率。最后, S^3 在后台运行监督器, 检测预测错误并调整已分配内存的大小, 使其更准确。通过集成这些组件, S^3 优化内存使用和调度, 从而在 GPU 上部署基于 Transformer 的文本生成 LLM 时最大化吞吐量。

LLM 部署场景分为在线与离线两类。聊天机器人 [7, 8] 等在线场景要求服务提供者在严格的时延服务等级目标 (SLO) 内生成序列。离线场景包括评分 [9]、数据整理 [10] 等应用, 时延 SLO 较宽松, 更重视吞吐量而非端到端时延。FasterTransformer [4] 和 xFormers [11] 则优先降低时延。我们认为, 只要时延已低于 SLO, 就没有必要继续优先追求更低时延。因此, 我们将 S^3 设计为在这些时延 SLO 下获得更高吞吐量。图 1 展示了 S^3 通过时延权衡能够获得多大的吞吐量提升。对于在线场景, 我们按英语读者的平均阅读速度设定 SLO, 即每秒 4 个单词 [12]、每个 token 0.1875 秒 [13]。左图中参数量小于 1000 亿的模型, 在所有可用批大小下都满足这一 SLO。该 SLO 为服务提供者提高这些模型的吞吐量留下了空间。事实上, 对于 LLAMA-33B [14], 甚至可以在不增加时延的情况下提高吞吐量。右图改变 GPU 数量, 展示了在时延 SLO 较宽松的离线场景中最大化吞吐量的机会。

我们从吞吐量和成本效率两个方面评估 S^3 , 分析涵盖离线与在线场景。在以平均阅读速度作为时延 SLO 的在线场景中, 我们发现, S^3 在遵守相同 SLO 的同时, 最多可生成 6.49 倍的序列。对于离线场景, 我们观察到, S^3 在不同模型上最高实现 $6.49\times$ 的加速。 S^3 不改变模型架构, 因而不影响模型的困惑度。此外, 我们评估了 S^3 的成本效率, 发现使用 6 张 GPU 的 S^3 可以获得与使用 10 张 GPU 的原始系统几乎相同的吞吐量。

本文的贡献如下:

- 我们在较宽松的时延 SLO 下增大了可用批大小, 使服务提供者能通过更大的批次, 在在线与离线场景中获得更高吞吐量。
- 我们微调 DistilBERT 模型, 根据输入提示预测输出序列长度。
- 我们提供了从预测错误中恢复的机制。 S^3 抢占内存使用超过分配额度的序列, 并重新训练预测器, 使其从错误中学习。

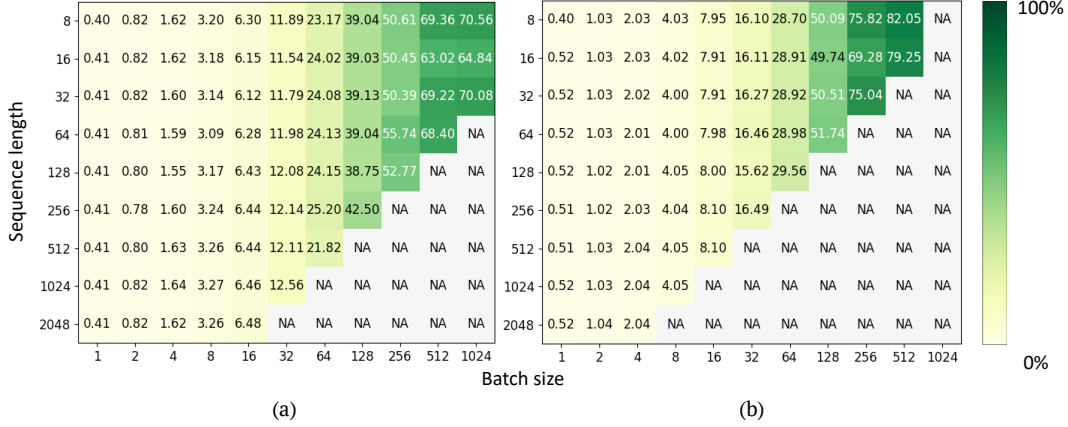


图 2: 在 NVIDIA A100 上, (a) GPT-J 与 (b) GPT-NEOX 的计算资源利用率。图内英文: Sequence length 为序列长度, Batch size 为批大小, NA 表示该配置无法运行, 色条表示计算资源利用率。

2 背景与动机

2.1 生成式人工智能模型

基于 Transformer 的生成式模型采用自回归方式, 根据此前的 token 预测最可能的下一个 token。模型每次生成一个 token, 因此要生成长度为 n 个 token 的序列, 就必须运行 n 次。一次迭代中, 输入 token 穿过整个模型; 模型由多层 Transformer 堆叠而成, 每层包含一个注意力层、两个层归一化层和两个前馈层。其中, 自注意力层利用此前 token 的信息生成下一个 token。

例如, 在第 i^{th} 次迭代中, 模型在自注意力层里让当前 token (t_i) 关注此前已生成的所有 token ($t_0, \dots, t_{i-1}$)。自注意力层可表示为:

$$h_{out} = softmax(\frac{q_i \cdot K^T}{\sqrt{d_h}}) \cdot V$$

其中, d_h 为模型的隐藏维度; $h_{out}, q_i \in \mathbb{R}^{d_h}$ 分别为输出隐藏向量和当前查询向量; $K, V \in \mathbb{R}^{i \times d_h}$ 为键矩阵和值矩阵。 K 和 V 矩阵的第 j^{th} 行分别表示 t_j 的键向量和值向量。两次点积使 t_i 关注当前序列中的所有键向量和值向量。

模型将 K 和 V 矩阵保存为键/值 (KV) 缓存, 避免每次迭代都重新生成键向量和值向量。否则, 模型需要保存此前每个 token 的隐藏状态, 并在每个 Transformer 层中将其与权重矩阵 $W_{QKV} \in \mathbb{R}^{d_h \times 2d_h}$ 相乘。原文将这部分开销记为每层 $2(i-1)d_h$ FLOPs, 认为它几乎等于在 t_i 处执行自注意力层所需的 $2id_h$ FLOPs。采用半精度数值时, KV 缓存每个 token 占用 $4ld_h$ 字节。键缓存和值缓存中的每个数均占 2 字节, 其中 l 为模型的 Transformer 层数。例如, GPT-NEOX [15] 包含 200 亿个参数、44 层、6144 个隐藏维度, 因此每个 token 的 KV 缓存约占 1 MB。¹

2.2 GPU 上的 KV 缓存管理

KV 缓存保存序列中此前 token 的信息, 因此序列较短时, 缓存相对较小, 例如只有几 MB, 容易放入 GPU 的高带宽内存 (HBM)。随着模型生成更多 token, 缓存不断增长。利用这一动态特性, Huggingface Transformers [3] (HF-Transformers) 持续为 KV 缓存分配更多内存, 并让计算暂停, 直到分配完成。这种方法使库能够为每份缓存分配精确的内存量, 代价是频繁访问内存。

¹译注: 此处保留原文的 FLOPs 表达式。按前述隐藏状态与稠密权重矩阵相乘的描述, 计算量通常还涉及隐藏维度的另一因子; 原文未说明这里的简化计数口径。

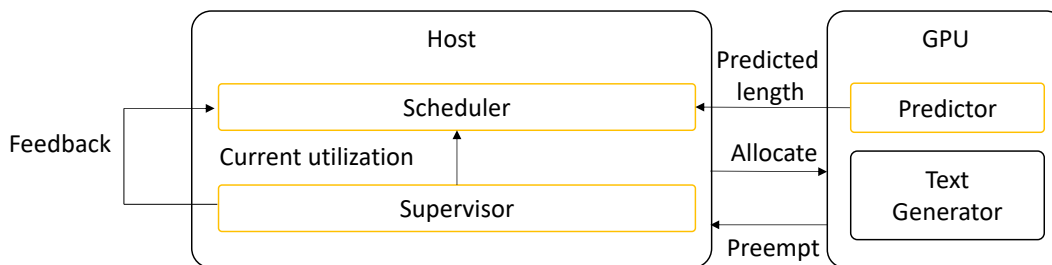


图 3: S^3 的概览。黄色方框表示 S^3 新增的组件。图内英文: Host 为主机, Scheduler 为调度器, Supervisor 为监督器, Predictor 为预测器, Text Generator 为文本生成器; Predicted length 为预测长度, Current utilization 为当前利用情况, Allocate 为分配, Preempt 为抢占, Feedback 为反馈。

为缓解这一问题, NVIDIA 的 FasterTransformer 库按最大序列长度为每条序列预留内存 [4, 16]。它只以追加方式填充已预留的内存, 从而去除冗余的内存访问。然而, 这种方法也有缺点: 为序列预留的内存超过实际需要。对于最大序列长度为 2048 个 token 的 GPT-NEOX, 即使某条序列最终只有 50 个 token, FasterTransformer 也会按 2048 个 token 预留空间。在一张 80 GB A100 上, 模型本身占 40 GB, 每条序列占 2.2 GB, 因此 FasterTransformer 对该模型可用的最大批大小小于 20。较小的批大小使 GPU 的大量计算资源得不到充分利用。即使存在利用率问题, 系统仍按最大序列长度预留内存, 是为了确保能够生成完整序列并改善用户体验。

2.3 观察

语言模型受内存限制 我们在一张 A100 GPU 上运行参数量为 60 亿的 GPT-J, 展示 GPU 资源利用不足的程度。该模型相对较小, GPU 能容纳更大批次和更长序列, 因此可以观察更宽范围的批大小与序列长度。图 2(a) 展示了改变批大小和序列长度时的 GPU 利用率。随着批大小增加, 利用率提高, 但最终会遭遇“内存悬崖”: 内存不足 (OOM) 错误使进程终止。以序列长度 1024 为例, 最多能容纳 8 条序列, 因此在这一长度下能达到的最大利用率为 12.19%。图 2(b) 展示了更大的 GPT-NEOX 模型因内存悬崖而出现的类似资源利用不足。该模型占用更多 GPU 内存, 因此在更小批大小、更短序列下就会遇到内存悬崖。请注意, 为了组批时避开内存悬崖, HF-Transformers 仍需提前知道输出长度。

这些图说明, 增大批大小可以提高神经网络的吞吐量。这种方法无需复杂优化, 只需将模型权重从 HBM 载入片上 SRAM 一次, 再让更多输入共享它。这样, GPU 可以启用闲置计算资源并同时处理多个输入。然而, 内存悬崖限制了更多资源的使用。下文将说明如何解决这一问题。

低效率的原因 HF-Transformers 的频繁内存分配和 FasterTransformer 受限的批大小, 都源于不知道生成序列的长度。如果能够知道确切的生成长度, 就可以为每条序列精确分配内存, 分别解决 HF-Transformers 的重复内存预留问题和 FasterTransformer 的不必要内存分配问题。

3 S^3 的设计

S^3 是一个系统与算法协同设计的框架, 通过预测序列长度来最大化 GPU 利用率, 从而提高吞吐量。如图 3 所示, S^3 包含三个组件: 1) 预测器; 2) 调度器; 3) 监督器。文本生成请求到达主机 DRAM 中的请求池后, 预测器预测其输出长度, 调度器依据预测结果对请求组批。调度器将批次派发到 GPU, 由文本生成模型生成批次内的文本。监督器监控 GPU 利用率并处理预测错误。本节介绍各组件及其交互方式。

输出序列长度预测器 我们利用预测器预测输出长度, 解决频繁分配和冗余分配内存的问题。具体而言, 我们微调一个原本用于序列分类的 DistilBERT [17] 模型, 将输出长度分类到相应的长度桶。我们对长度进行分桶, 是因为机器学习模型更擅长将候选范围缩小到最后一小段, 而不擅长在“最后一公里”进行精确搜索。每个桶覆盖的范围为 $\frac{\text{最大序列长度}}{\text{桶数}}$, 本文使用 10 个桶。

为此，我们在代表性的问答数据集 Alpaca [18] 上微调模型，以问题作为输入，以答案长度作为标签。预测器正确预测长度桶的准确率为 98.61%。错误预测与正确桶之间的平均距离为 1.03；原文据此认为，当桶宽加倍时，误差会趋近于 0。² 我们还在 Google Natural Questions 数据集 [19] 上微调预测器，得到 77.13% 的准确率；较小误差出现得比大误差更频繁。为使评估更完整，我们在非问答数据集 Pile [20] 上微调模型，得到 65.6% 的准确率。相对于仅有 10% 准确率的随机猜桶，预测器的准确率明显较高。

我们选择包含 6600 万参数的 DistilBERT，因为它规模较小、推理较快。其模型大小几乎可以忽略，甚至小于十亿参数级模型中的单个 Transformer 层，例如 60 亿参数 GPT-J [21] 的单层就包含 2.14 亿参数。时延也很小：请求到达服务器后，预测器只运行一次，而文本生成模型生成长度为 n 个 token 的输出需要运行 n 次。在 NVIDIA A100 上，DistilBERT 的测得运行时间为 3.7 ms，而文本生成模型需要数秒。

感知长度的序列调度器 调度器根据预测结果对序列组批并调度，在不超过 GPU HBM 容量的前提下最大化 GPU 利用率。这一问题可以表述为只有一个箱子的装箱问题变体：箱子的容量约束是 HBM 容量，每个物品的重量是相应序列的 KV 缓存大小。

我们采用递减首次适应算法 [22]，因为它简单。调度器首先将请求池中待调度的序列按预测长度递减排序。随后遍历请求池，检查当前序列的 KV 缓存是否不超过剩余 HBM；若可以容纳，就将它加入当前批次，并从可用 HBM 中减去其 KV 缓存大小。这个过程持续到没有可用 HBM，或已遍历整个请求池。

生成的批次形状不规则，其中有些序列比其他序列长。现有框架要么不支持这类批次 [3, 4]，要么支持但性能有限 [23]。不支持的框架会用填充 token 补齐较短序列，使同一批次内的长度一致；填充 token 不包含有用信息，因此浪费计算和内存。ORCA [16] 提出了一种解决方法：它发现，Transformer 某些层（例如前馈层）的输入形状与序列长度无关，因此可直接组批，无需填充短序列。对于要求输入形状一致的层，即自注意力层，ORCA 则串行处理输入，而不进行批处理。ORCA 表明，这对时延的影响可以忽略，因为自注意力层的输入不共享权重，也无法从组批中获得相同收益。因此，我们沿用 ORCA 的选择性组批技术。

S^3 还借用了 ORCA 的迭代级调度技术：无需等当前批次中的所有序列都生成完毕，而是在每次迭代后检查是否有序列完成。这提高了调度灵活性，去除了不必要的等待。最后，若模型无法放入单张 GPU， S^3 采用流水线并行，以 Transformer 层为粒度对模型分片。

监督器 监督器负责监控 GPU 利用率并处理预测错误。它在后台检查 HBM 可用空间，将信息传给调度器。若剩余内存足以容纳一条序列，调度器就把它加入正在运行的批次。

监督器也负责处理预测错误。若预测长度过短，它会抢占超过预留内存额度的序列：监控当前输出长度，在序列尚未结束却已经用完预留内存时将其逐出。监督器异步地把这些序列的当前状态，包括 KV 缓存和已生成的 token，迁回请求池，并释放 GPU 内存。此时 K、V 矩阵出现碎片，原来保存被逐出缓存的位置留下空行。监督器移动空行下方的行，使所有行连续存储。这是现有库 [24, 25] 要求的内存格式，也解决了碎片问题。最后，监督器将被逐出序列的内存额度加倍，以修正偏短预测。

最后，监督器持续在后台训练预测器，用预测错误的序列进行训练，使其从错误中学习。训练时间相对较短：我们的测量表明，每次训练迭代平均需要 11 ms，而序列生成需要数秒或更久。原文据此认为，即使训练预测器 10 个 epoch，重新训练的开销也低于 10%。³

整体流程 下面汇总 S^3 如何使用各组件处理序列生成请求。首先，文本生成请求进入请求池；预测器预测池中序列的输出长度；监督器在后台检查当前 HBM 使用情况。随后，调度器结合长度预测和可用 HBM 对请求组批，使 GPU 利用率最大化。最后，调度器把批次送往 GPU，生成这些已调度的序列。

表 1: 评估中使用的模型架构

模型	参数量	层数	模型维度	注意力头数
GPT-J [21]	6B	28	5120	16
LLAMA 13B [14]	13B	40	4096	40
GPT-NEOX [15]	20B	44	6144	64
LLAMA 33B [14]	30B	60	6656	52
GPT3 175B [26]	175B	96	12288	96

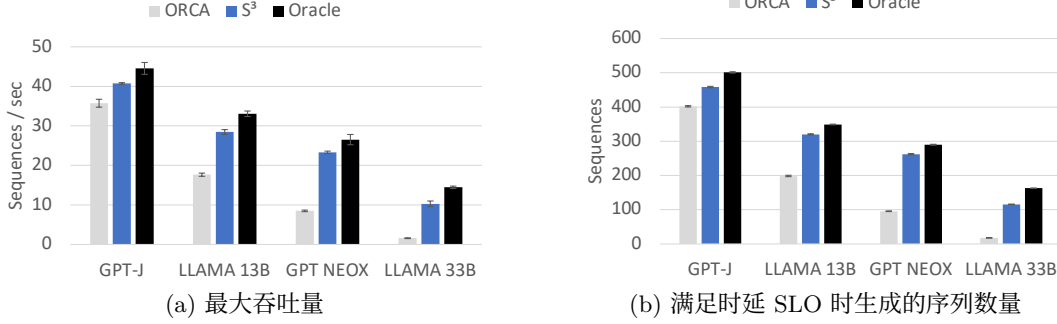


图 4: 不同模型的时延与吞吐量。图内英文: Sequences / sec 为每秒序列数, Sequences 为序列数量; ORCA、S³ 和 Oracle 分别为基线、本文系统、具有完美预测器的理想系统。

4 评估

我们展示 S³ 如何通过预测输出长度提高吞吐量, 也展示它如何使用更少 GPU 来降低模型服务成本。

环境 我们在 Lenovo ThinkSystem SD650-N V2 服务器 [27] 上评估, 使用一张 80 GB NVIDIA A100 GPU, 通过 PCIe 4.0×8 与主机 DRAM 连接。

基线 我们与 ORCA [16] 比较; 该 Transformer 服务系统通过迭代级调度与选择性组批提高吞吐量。在不知道输出长度时, ORCA 必须按最大序列长度分配内存, 保证完整序列生成。S³ 则预测输出长度并据此分配内存。我们在 FasterTransformer [4] 上实现这些系统, 因为其优化更多, 速度快于 HF-Transformers [3]。我们还将 S³ 与一个具有完美预测器的理想系统比较, 并将后者称为 Oracle。

模型 评估使用参数量从 60 亿到 1750 亿的模型, 具体配置见表 1。

4.1 吞吐量分析

我们使用流行的开源问答数据集 Alpaca [18] 评估 S³ 的吞吐量, 向 S³ 提交问题并要求生成答案。

离线场景: 最大吞吐量 图 4(a) 报告不同模型的最大吞吐量, 单位为每秒序列数; 每种配置均采用其最大批大小进行测量。S³ 的吞吐量为 ORCA 的 1.13× 至 6.49×, 接近 Oracle, 但与 Oracle 仍相差 9.34% 至 40.52%。模型越大, 差距越大, 因为大部分 HBM 用于保存权重, 即使 S³ 的批大小也受限, 见表 2。如图 1 所示, S³ 在吞吐量达到饱和之前就遇到了批大小上限。

可以看到, 最大吞吐量提高至 6.49×, 而各模型的批大小增加了近 10×。这源于 Transformer 的特点: 前馈层能从组批中受益, 自注意力层则不能。前馈层的输入共享权重, 自注意力层

²译注: 平均错桶距离为 1.03 本身不能保证桶宽加倍后误差趋于零; 是否减少错桶还取决于桶边界及误差分布。这里保留作者的判断, 不把它视为已证明的保证。

³译注: 一次训练迭代与一个 epoch 并不相同; 总开销还取决于每个 epoch 的迭代数及后台训练与推理的资源竞争。原文未给出据此计算完整重训练开销所需的全部条件。

表 2: 不同模型的平均批大小和迭代次数

模型	ORCA		S ³		Oracle	
	批大小	迭代次数	批大小	迭代次数	批大小	迭代次数
GPT-J	69.94	7988	530	1054	564.88	989
LLAMA 13B	31.61	17675	274.66	2034	527.04	1060
GPT-NEOX	16.89	33069	157.59	3545	292.19	1912
LLAMA 33B	4	139790	42.47	13155	77.35	7223

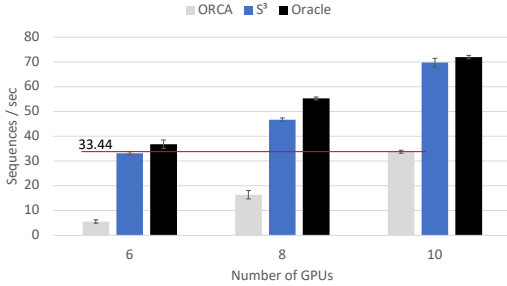


图 5: GPT-3 在不同 GPU 数量下的最大吞吐量。图内英文: Sequences / sec 为每秒序列数, Number of GPUs 为 GPU 数量; 红线标出 33.44 序列/秒。

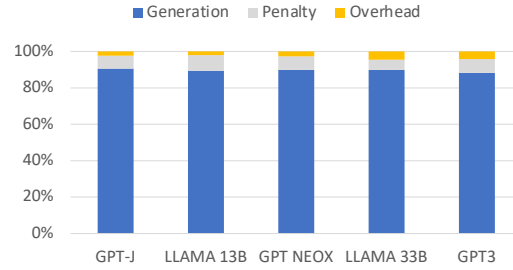


图 6: S³ 的时延构成。图内英文: Generation 为生成时间, Penalty 为抢占迁出与恢复的惩罚时间, Overhead 为预测、调度与监督的额外开销; 纵轴表示各项时间占比。

的输入只关注各自序列。当可并行部分相对于串行部分更大时, S³ 的收益更高; 然而, 增大批大小也会加入更多串行输入, 使串行部分变大。GPT-J 是这一特点的例子: 从 ORCA 到 S³ 的吞吐量增加 1.13 $\times$, 从 S³ 到 Oracle 增加 1.09 $\times$, 而原文称相应批大小相差 8.69 $\times$ 和 1.97 $\times$ 。⁴

在线场景: 考虑 SLO 的吞吐量 下面考虑带有时延 SLO 约束的场景。英语读者平均每秒阅读 4 个单词 [12], 即每词 0.25 秒; 按每 token 0.75 个单词 [13] 计算, 我们将生成一个 token 的 SLO 设为 0.1875 秒。随后以 0.1875 乘以序列长度, 得到每条序列的时延 SLO, 例如 60 个 token 对应 11.25 秒。

图 4b 报告每个模型使用 ORCA、S³、Oracle 时生成的序列数。Oracle 在 GPT-J、LLAMA 13B、GPT-NEOX 上采用最大批大小时会超过 SLO, 因此本项评估限制了这些模型的 Oracle 批大小。S³ 生成的序列数量接近理想情况, 为 ORCA 的 1.13 $\times$ 至 6.49 $\times$ 。这一提升与离线场景相似, 因为 S³ 在多数情况下采用最大批大小也能满足 SLO。然而, 由于限制了 Oracle 的批大小及吞吐量, S³ 与 Oracle 的差距相对于离线场景减少了 10%。

4.2 成本分析

我们在不同 GPU 数量下评估 S³。对 6 张、8 张 GPU 的配置, 以流水线并行方式划分 GPT-3, 每张 GPU 分别分配 16 层、12 层 Transformer。对 10 张 GPU 的配置, 9 张各分配 10 层, 剩余一张分配 6 层。S³ 对各批次执行流水线处理; 当处理第一分片的 GPU 把结果交给下一流水线阶段后, 便调度一个新批次。让各 GPU 同时处理不同批次, 可以减少闲置时间。

图 5 报告不同 GPU 数量下的最大吞吐量。首先, 使用 6 张 GPU 的 S³ 与使用 10 张 GPU 的 ORCA 吞吐量相近; 具体来说, ORCA 仅高出 0.92%。更多 GPU 将模型划分得更细, 为 KV 缓存留下更多空间, 因此可以增大批大小。表 3 中 GPU 数量增加时批大小也增加, 支持了这一解释。S³ 通过优化内存分配策略, 可以使用更少 GPU 获得类似效果。

由此自然产生与吞吐量评估类似的问题: 使用 6 张 GPU 的 S³ 批大小为使用 10 张 GPU 的 ORCA 的 2.33 $\times$, 为什么吞吐量仍然相近? 原因在于流水线执行, 以及 Transformer 自

⁴译注: 此处两个批大小倍率与上表 GPT-J 行不一致。该行数值为 69.94、530、564.88, 相邻比值约为 7.58 与 1.07; 8.69 和 1.97 更接近表中 LLAMA 13B 行的相邻比值。原文数值均予保留。

表 3: 不同系统配置的平均批大小和迭代次数

GPU 数量	ORCA		S ³		Oracle	
	批大小	迭代次数	批大小	迭代次数	批大小	迭代次数
6	11.95	46734	114.22	4891	209.47	2667
8	28.68	19482	247.85	2254	470.65	1187
10	48.99	11403	399.62	1398	564.88	989

注意力层的串行特性，见 4.1。在流水线稳定运行时，系统每经过 $\lceil \frac{l}{\text{GPU 数量}} \rceil$ 层就完成一个批次，而非每经过 l 层。例如，对 GPT-3，使用 6 张 GPU 的 S³ 每经过 16 层即可完成一个批次，而不是 96 层；使用 10 张 GPU 的 ORCA 每经过 10 层完成一个批次，因此处理更快。增加的时延抵消了 S³ 的吞吐量收益，使两系统几乎具有相同吞吐量，尽管 S³ 的批大小是其 2.33 $\times$ 。

4.3 开销：时延构成

我们评估 S³ 各组件的运行时延，将其分为三类：生成时间、惩罚时间和额外开销。生成时间是 S³ 运行模型生成 token 所花的时间；惩罚时间是将 KV 缓存和隐藏状态从 GPU 抢占迁出、再载入 GPU 所花的时间；额外开销包括预测器、调度器和监督器的总时间。图 6 表明，惩罚时间与额外开销之和相对于生成时间较小，平均为 11%。如果预测器更不准确、引起更多 GPU 与主机之间的数据传输，惩罚时间会增加。如果使用更准确但更重的预测器，额外开销则会增加。只要预测器适当，当文本生成模型生成更长序列时，这些开销便可得到摊薄。

5 相关工作

机器学习服务系统 机器学习受到广泛关注，带动了对其服务平台的大量研究 [1, 4, 6, 16, 28–38]。基于 Transformer 的语言模型表现突出，尤其促使研究者开发专门面向 Transformer 的服务系统 [4, 6, 34, 37, 38]。多数系统侧重降低时延，较少考虑吞吐量；例外包括 [6, 31, 39]。这些面向吞吐量的系统利用内存层级，将部分模型存入较慢的存储，借此增大批大小。然而，它们为每条序列分配同样数量的内存，也不考虑根据预测抢占序列。S³ 可以通过减少所需内存改善这些系统，降低对 SSD 等容量更大但速度更慢的存储的需求，并减少总体内存成本。

降低 KV 缓存开销 Transformer 注意力层的计算和内存需求随序列长度呈二次增长，这一问题已得到广泛研究。低秩近似 [40, 41] 和利用稀疏性 [42–44] 等方法旨在缓解这一问题。另一种方法称为多查询注意力 [37, 45]：不再像表 1 所示那样使用多个注意力头，而是让 KV 缓存采用一个注意力头，减少缓存大小。此外，通过压缩 [46]、量化 [6, 47, 48] 减小模型的工作，也能帮助减小 KV 缓存。这些方法与本文互补，可以减少预测错误引起的惩罚，使系统能够采用准确率稍低但更轻量的预测器。

序列长度预测 根据输入序列预测输出长度的工作还较少。Yan 等人 [49] 提出了带有嵌入层的小型卷积网络，用来预测机器翻译的输出长度。我们采用类似思路，但使用小型 Transformer 预测器来估计文本生成任务的输出长度，从而为文本生成工作负载提供较准确的长度预测。

6 局限性与结论

局限性 本文采用以下假设。由于没有公开的文本生成请求轨迹，我们假设请求轨迹与公开问答数据集相似；分析实际轨迹将有助于把 S³ 部署到商业工作负载中。类似地，文本生成是一项较新的服务，还没有像其他机器学习工作负载 [50] 那样具有标准化时延 SLO，因此我们采用英语读者的平均阅读速度作为 SLO。如果组织公开不同文本生成应用的 SLO，就能在更广泛的场景中评估 S³。

结论 本文提出 S^3 , 一个旨在为基于 Transformer 的生成式模型提供高吞吐量服务的框架。 S^3 用预测器估计生成序列的输出长度, 再据此调度序列, 使吞吐量最大化; 同时处理可能的预测错误, 确保运行可靠。通过为不同输入分配不同大小的内存, S^3 体现了不同序列应区别处理的思想。这种方法扩展了传统时延与吞吐量的权衡边界, 为优化这一权衡提供了新的可能。

参考文献

- [1] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, “Clipper: A Low-Latency online prediction serving system,” in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. Boston, MA: USENIX Association, Mar. 2017, pp. 613–627. [Online]. Available: <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/crankshaw>
- [2] R. Chard, Z. Li, K. Chard, L. Ward, Y. Babuji, A. Woodard, S. Tuecke, B. Blaiszik, M. J. Franklin, and I. Foster, “Dlhub: Model and data serving for science,” in *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2019, pp. 283–292.
- [3] H. Face, “Transformers,” <https://github.com/huggingface/transformers>.
- [4] NVIDIA, “FasterTransformer,” <https://github.com/NVIDIA/FasterTransformer>.
- [5] OpenAI, “OpenAI research GPT4,” <https://openai.com/research/gpt-4>.
- [6] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, D. Y. Fu, Z. Xie, B. Chen, C. Barrett, J. E. Gonzalez, P. Liang, C. Ré, I. Stoica, and C. Zhang, “High-throughput Generative Inference of Large Language Models with a Single GPU,” *arXiv e-prints*, p. arXiv:2303.06865, Mar. 2023.
- [7] Google, “Bard,” <https://bard.google.com/>.
- [8] OpenAI, “ChatGPT,” <https://chat.openai.com/>.
- [9] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar, B. Newman, B. Yuan, B. Yan, C. Zhang, C. Cosgrove, C. D. Manning, C. Ré, D. Acosta-Navas, D. A. Hudson, E. Zelikman, E. Durmus, F. Ladhak, F. Rong, H. Ren, H. Yao, J. Wang, K. Santhanam, L. Orr, L. Zheng, M. Yuksekgonul, M. Suzgun, N. Kim, N. Guha, N. Chatterji, O. Khattab, P. Henderson, Q. Huang, R. Chi, S. M. Xie, S. Santurkar, S. Ganguli, T. Hashimoto, T. Icard, T. Zhang, V. Chaudhary, W. Wang, X. Li, Y. Mai, Y. Zhang, and Y. Koreeda, “Holistic Evaluation of Language Models,” *arXiv e-prints*, p. arXiv:2211.09110, Nov. 2022.
- [10] A. Narayan, I. Chami, L. Orr, S. Arora, and C. Ré, “Can Foundation Models Wrangle Your Data?” *arXiv e-prints*, p. arXiv:2205.09911, May 2022.
- [11] B. Lefauveux, F. Massa, D. Liskovich, W. Xiong, V. Caggiano, S. Naren, M. Xu, J. Hu, M. Tintore, S. Zhang, P. Labatut, and D. Haziza, “xformers: A modular and hackable transformer modelling library,” <https://github.com/facebookresearch/xformers>, 2022.
- [12] M. Brysbaert, “How many words do we read per minute? a review and meta-analysis of reading rate,” *Journal of Memory and Language*, vol. 109, p. 104047, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0749596X19300786>
- [13] OpenAI, “OpenAI API Documentation,” <https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them>.
- [14] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [15] S. Black, S. Biderman, E. Hallahan, Q. Anthony, L. Gao, L. Golding, H. He, C. Leahy, K. McDonell, J. Phang, M. Pieler, U. S. Prashanth, S. Purohit, L. Reynolds, J. Tow, B. Wang, and S. Weinbach, “GPT-NeoX-20B: An open-source autoregressive language model,” in *Proceedings of BigScience Episode #5 – Workshop on Challenges & Perspectives in Creating Large Language Models*. virtual+Dublin: Association for Computational Linguistics, May 2022, pp. 95–136. [Online]. Available: <https://aclanthology.org/2022.bigscience-1.9>

- [16] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for Transformer-Based generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/yu>
- [17] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” *arXiv e-prints*, p. arXiv:1910.01108, Oct. 2019.
- [18] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, “Stanford alpaca: An instruction-following llama model,” https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [19] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, M. Kelcey, J. Devlin, K. Lee, K. N. Toutanova, L. Jones, M.-W. Chang, A. Dai, J. Uszkoreit, Q. Le, and S. Petrov, “Natural questions: a benchmark for question answering research,” 2019.
- [20] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, S. Presser, and C. Leahy, “The Pile: An 800GB Dataset of Diverse Text for Language Modeling,” p. arXiv:2101.00027, Dec. 2020.
- [21] B. Wang and A. Komatsuzaki, “GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model,” <https://github.com/kingoflolz/mesh-transformer-jax>, May 2021.
- [22] D. S. Johnson, “Near-optimal bin packing algorithms,” Ph.D. dissertation, Massachusetts Institute of Technology, 1973.
- [23] PyTorch, “Pytorch Nested Tensor,” <https://pytorch.org/docs/stable/nested.html>.
- [24] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>
- [25] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard *et al.*, “Tensorflow: A system for large-scale machine learning,” in *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 16)*, 2016, pp. 265–283.
- [26] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Nee-lakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language Models are Few-Shot Learners,” *arXiv e-prints*, p. arXiv:2005.14165, May 2020.
- [27] Lenovo, “Lenovo ThinkSystem SD650-N V2 Server,” <https://lenovopress.lenovo.com/lp1396-thinksystem-sd650-n-v2-server>.
- [28] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis, “INFaaS: Automated model-less inference serving,” in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, Jul. 2021, pp. 397–411. [Online]. Available: <https://www.usenix.org/conference/atc21/presentation/romero>
- [29] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, “Serving DNNs like clockwork: Performance predictability from the bottom up,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 443–462. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/gujarati>
- [30] S. Choi, S. Lee, Y. Kim, J. Park, Y. Kwon, and J. Huh, “Serving heterogeneous machine learning models on Multi-GPU servers with Spatio-Temporal sharing,” in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*.

- Carlsbad, CA: USENIX Association, Jul. 2022, pp. 199–216. [Online]. Available: <https://www.usenix.org/conference/atc22/presentation/choi-seungbeom>
- [31] R. Yazdani Aminabadi, S. Rajbhandari, M. Zhang, A. A. Awan, C. Li, D. Li, E. Zheng, J. Rasley, S. Smith, O. Ruwase, and Y. He, “DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale,” *arXiv e-prints*, p. arXiv:2207.00032, Jun. 2022.
 - [32] U. Gupta, S. Hsia, V. Saraph, X. Wang, B. Reagen, G.-Y. Wei, H.-H. S. Lee, D. Brooks, and C.-J. Wu, “DeepRecSys: A System for Optimizing End-To-End At-scale Neural Recommendation Inference,” *arXiv e-prints*, p. arXiv:2001.02772, Jan. 2020.
 - [33] S. Hsia, U. Gupta, B. Acun, N. Ardalani, P. Zhong, G.-Y. Wei, D. Brooks, and C.-J. Wu, “Mp-rec: Hardware-software co-design to enable multi-path recommendation,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 449–465. [Online]. Available: <https://doi.org/10.1145/3582016.3582068>
 - [34] J. Fang, Y. Yu, C. Zhao, and J. Zhou, “TurboTransformers: An Efficient GPU Serving System For Transformer Models,” *arXiv e-prints*, p. arXiv:2010.05680, Oct. 2020.
 - [35] C.-F. Wu, C.-J. Wu, G.-Y. Wei, and D. Brooks, “A joint management middleware to improve training performance of deep recommendation systems with ssds,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC 22)*, 2022, pp. 157–162.
 - [36] C. Olston, N. Fiedel, K. Gorovoy, J. Harmsen, L. Lao, F. Li, V. Rajashekhar, S. Ramesh, and J. Soyke, “TensorFlow-Serving: Flexible, High-Performance ML Serving,” *arXiv e-prints*, p. arXiv:1712.06139, Dec. 2017.
 - [37] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean, “Efficiently scaling transformer inference,” *arXiv preprint arXiv:2211.05102*, 2022.
 - [38] X. Wang, Y. Xiong, Y. Wei, M. Wang, and L. Li, “LightSeq: A High Performance Inference Library for Transformers,” *arXiv e-prints*, p. arXiv:2010.13887, Oct. 2020.
 - [39] Huggingface, “Huggingface Accelerate,” <https://huggingface.co/docs/accelerate/index>.
 - [40] K. Choromanski, V. Likhoshesterov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser, D. Belanger, L. Colwell, and A. Weller, “Rethinking Attention with Performers,” *arXiv e-prints*, p. arXiv:2009.14794, Sep. 2020.
 - [41] V. Likhoshesterov, K. Choromanski, J. Davis, X. Song, and A. Weller, “Sub-Linear Memory: How to Make Performers SLiM,” *arXiv e-prints*, p. arXiv:2012.11346, Dec. 2020.
 - [42] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The Long-Document Transformer,” *arXiv e-prints*, p. arXiv:2004.05150, Apr. 2020.
 - [43] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, “Big Bird: Transformers for Longer Sequences,” *arXiv e-prints*, p. arXiv:2007.14062, Jul. 2020.
 - [44] H. Wang, Z. Zhang, and S. Han, “SpAtten: Efficient Sparse Attention Architecture with Cascade Token and Head Pruning,” *arXiv e-prints*, p. arXiv:2012.09852, Dec. 2020.
 - [45] N. Shazeer, “Fast Transformer Decoding: One Write-Head is All You Need,” *arXiv e-prints*, p. arXiv:1911.02150, Nov. 2019.
 - [46] J. W. Rae, A. Potapenko, S. M. Jayakumar, and T. P. Lillicrap, “Compressive Transformers for Long-Range Sequence Modelling,” *arXiv e-prints*, p. arXiv:1911.05507, Nov. 2019.
 - [47] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, “I-BERT: Integer-only BERT Quantization,” *arXiv e-prints*, p. arXiv:2101.01321, Jan. 2021.

- [48] Z. Liu, Y. Wang, K. Han, W. Zhang, S. Ma, and W. Gao, “Post-training quantization for vision transformer,” in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 28 092–28 103. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2021/file/ec8956637a99787bd197eacd77acce5e-Paper.pdf
- [49] Z. Yang, Y. Gao, W. Wang, and H. Ney, “Predicting and using target length in neural machine translation,” in *Proceedings of the 1st Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 10th International Joint Conference on Natural Language Processing*. Suzhou, China: Association for Computational Linguistics, Dec. 2020, pp. 389–395. [Online]. Available: <https://aclanthology.org/2020.aacl-main.41>
- [50] V. Janapa Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J. S. Gardner, I. Hubara, S. Idgunji, T. B. Jablin, J. Jiao, T. St. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A. Tejusve Raghunath Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou, “MLPerf Inference Benchmark,” *arXiv e-prints*, p. arXiv:1911.02549, Nov. 2019.